LLM Integration — Change Documentation

Project: 3D Calculus Visualizer
LLM Host: Google Colab (tunnelled via ngrok)
Goal: A math-tutor LLM running on Colab explains concepts and emits structured equation payloads consumed by the visualizer — with zero direct control over the GUI.

Architecture Overview

```
┌──────────────────────────────────────────────────────────────┐
│  GOOGLE COLAB  (GPU runtime)                                 │
│                                                              │
│  ┌──────────────────┐    ┌──────────────────────────────┐   │
│  │  Tuned LLM       │──▶│  FastAPI server  (colab_server)  │   │
│  │  (HF / GGUF)     │    │  POST /ask  →  GET /payload  │   │
│  └──────────────────┘    └──────────────────────────────┘   │
│                                      │                       │
│                                      │                       │
│                              ngrok tunnel                    │
│                        https://xxxx.ngrok-free.app           │
└──────────────────────────────────────┬──────────────────────┘
                                       │  HTTP (JSON)
                                       ▼
┌──────────────────────────────────────────────────────────────┐
│  LOCAL MACHINE                                               │
│                                                              │
│  chat_panel.py ──▶ llm_bridge.py ──▶ viz_bus.py ──▶ gui.py   │
│  (chat UI)         (HTTP client)     (queue)       (visualizer)│
└──────────────────────────────────────────────────────────────┘
```

Key constraint honoured: `llm_bridge.py` never imports `gui.py`.
It only produces `VisualizationPayload` objects pushed onto `viz_bus`.
The LLM on Colab never touches the GUI — it only returns JSON.

Part A — Google Colab Integration Script

Create a notebook on Colab and run the cells below in order.

Cell 1 — Install dependencies

```python
```

```
!pip install -q fastapi uvicorn pyngrok nest-asyncio
!pip install -q transformers accelerate bitsandbytes torch
# If using GGUF / llama.cpp models:
# !pip install -q llama-cpp-python
```

Cell 2 — Load your tuned LLM

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

MODEL_ID = "your-hf-username/your-math-tutor-model"  # replace

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16,
    device_map="auto",          # uses GPU automatically
    load_in_4bit=True,          # remove if your model is already small
)
model.eval()
print("Model loaded.")
```

> **If using a GGUF model via llama-cpp-python**, replace Cell 2 with:
>
> ```python
> from llama_cpp import Llama
> model = Llama(model_path="/content/your-model.gguf", n_gpu_layers=-1)
> ```

Cell 3 — System prompt & generation helper

The system prompt is the **only contract** between the LLM and the visualizer.
It must be reproduced exactly — the local app parses this JSON blindly.

```python
```

```
SYSTEM_PROMPT = """You are a university-level mathematics tutor specialising
in multivariable calculus.

When the user asks about any mathematical concept, respond with ONLY a JSON
object — no markdown fences, no extra text — matching this exact schema:

{
  "concept_title": "<short title, e.g. Saddle Point>",
  "equation": "<f(x,y) in Python/SymPy syntax>",
  "x_range": [<float min>, <float max>],
  "y_range": [<float min>, <float max>],
  "step": <float between 0.05 and 0.5>,
  "explanation": "<formal multi-paragraph plain-text explanation>"
}

Allowed symbols in equation: x y sin cos tan exp log sqrt Abs pi e
sinh cosh tanh ceiling floor  — use ** for powers, standard arithmetic only.

Choose x_range/y_range/step so the most interesting region of the surface
is clearly visible.

explanation must cover: definition, geometric intuition, key properties,
and what the partial derivatives and critical points reveal.

Never output anything outside the JSON object."""


def generate(user_text: str, max_new_tokens: int = 512) -> str:
    """Run inference and return the raw model output string."""
    prompt = f"<|system|>\n{SYSTEM_PROMPT}\n<|user|>\n{user_text}\n<|assistant|>\n"
    # Adjust the chat template tokens to match your model's format.
    # Common alternatives:
    #   "[INST] <<SYS>>\n{system}\n<</SYS>>\n{user} [/INST]"  ← Llama-2
    #   "<|im_start|>system\n{system}<|im_end|>\n..."          ← ChatML

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,          # greedy — keeps JSON well-formed
            temperature=1.0,
            pad_token_id=tokenizer.eos_token_id,
```

```
    )
    # Decode only the newly generated tokens
    new_ids = output_ids[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(new_ids, skip_special_tokens=True).strip()
```

Cell 4 — FastAPI server + ngrok tunnel

```python
```

```python
import asyncio, json, threading, queue
import nest_asyncio
import uvicorn
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from pyngrok import ngrok

nest_asyncio.apply()

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                   allow_methods=["*"], allow_headers=["*"])

# Thread-safe queue: generation results wait here
_result_queue: queue.Queue = queue.Queue()

class AskRequest(BaseModel):
    text: str

class PayloadResponse(BaseModel):
    status: str
    data: dict | None = None
    error: str | None = None


@app.post("/ask")
async def ask_endpoint(req: AskRequest):
    """
    Accepts a user question, runs the LLM in a background thread,
    deposits the parsed JSON result into _result_queue.
    Returns immediately with {"status": "queued"}.
    """
    def _run():
        try:
            raw = generate(req.text)
            # Strip accidental markdown fences if model added them
            raw = raw.strip().lstrip("```json").lstrip("```").rstrip("```").strip()
            parsed = json.loads(raw)
            _result_queue.put({"status": "ok", "data": parsed})
        except json.JSONDecodeError as e:
            _result_queue.put({"status": "error", "error": f"JSON parse failed: {e}
        except Exception as e:
```

```
            _result_queue.put({"status": "error", "error": str(e)})

    threading.Thread(target=_run, daemon=True).start()
    return {"status": "queued"}


@app.get("/payload")
async def payload_endpoint():
    """
    Polls the result queue.
    Returns {"status": "pending"} if generation is not done yet,
    or {"status": "ok", "data": {...}} when it is.
    """
    try:
        result = _result_queue.get_nowait()
        return result
    except queue.Empty:
        return {"status": "pending"}


@app.get("/health")
async def health():
    return {"status": "alive"}


# ── Start ngrok tunnel ──────────────────────────────────────────
NGROK_AUTH_TOKEN = "YOUR_NGROK_AUTHTOKEN"   # paste from ngrok dashboard
ngrok.set_auth_token(NGROK_AUTH_TOKEN)

tunnel = ngrok.connect(8000)
PUBLIC_URL = tunnel.public_url
print("=" * 60)
print(f"  Colab server live at:  {PUBLIC_URL}")
print(f"  POST {PUBLIC_URL}/ask      — send a question")
print(f"  GET  {PUBLIC_URL}/payload  — poll for result")
print(f"  GET  {PUBLIC_URL}/health   — connectivity check")
print("=" * 60)

# ── Run server (blocking — this cell stays running) ──────────────
uvicorn.run(app, host="0.0.0.0", port=8000)
```

> Copy the printed `PUBLIC_URL` — you will paste it into `llm_bridge.py` on your local machine.

Part B — Local Project Changes

New file: `viz_bus.py` *(~25 lines)*

```python
# viz_bus.py
import queue
from dataclasses import dataclass

@dataclass
class VisualizationPayload:
    concept_title: str
    equation:      str
    x_range:       tuple[float, float]
    y_range:       tuple[float, float]
    step:          float
    explanation:   str

_bus: queue.Queue = queue.Queue()

def publish(payload: VisualizationPayload) -> None:
    _bus.put(payload)

def subscribe() -> queue.Queue:
    return _bus
```

New file: `llm_bridge.py` *(~80 lines)*

This is now a pure HTTP client. It knows nothing about the GUI.

```python
```

```python
# llm_bridge.py
import threading, time, requests
import viz_bus as bus
from viz_bus import VisualizationPayload

# ── Configuration ──────────────────────────────────────────────
COLAB_BASE_URL = "https://xxxx.ngrok-free.app"   # ← paste your ngrok URL
POLL_INTERVAL  = 1.5    # seconds between /payload polls
POLL_TIMEOUT   = 120    # seconds before giving up


def _validate_payload(data: dict) -> VisualizationPayload:
    """Strict field validation before the visualizer ever sees the data."""
    required = {"concept_title", "equation", "x_range", "y_range",
                "step", "explanation"}
    missing = required - data.keys()
    if missing:
        raise ValueError(f"LLM response missing fields: {missing}")

    xr = tuple(float(v) for v in data["x_range"])
    yr = tuple(float(v) for v in data["y_range"])
    step = float(data["step"])

    if len(xr) != 2 or xr[0] >= xr[1]:
        raise ValueError(f"Invalid x_range: {xr}")
    if len(yr) != 2 or yr[0] >= yr[1]:
        raise ValueError(f"Invalid y_range: {yr}")
    if not (0.01 <= step <= 1.0):
        raise ValueError(f"Step out of bounds: {step}")

    return VisualizationPayload(
        concept_title = str(data["concept_title"]),
        equation      = str(data["equation"]),
        x_range       = xr,
        y_range       = yr,
        step          = step,
        explanation   = str(data["explanation"]),
    )


def ask(user_text: str, on_status=None, on_error=None) -> None:
    """
    Non-blocking. Sends the question to Colab, polls until result
```

```
    arrives, validates it, then publishes to viz_bus.
    on_status(msg) — called with progress strings (optional)
    on_error(msg)  — called on failure (optional)
    """
    def _worker():
        def _status(msg):
            if on_status:
                on_status(msg)

        def _error(msg):
            if on_error:
                on_error(msg)

        try:
            # 1. Send query to Colab
            _status("Sending question to Colab…")
            resp = requests.post(
                f"{COLAB_BASE_URL}/ask",
                json={"text": user_text},
                timeout=15,
            )
            resp.raise_for_status()

            # 2. Poll /payload until result is ready
            _status("LLM is thinking…")
            deadline = time.time() + POLL_TIMEOUT
            while time.time() < deadline:
                time.sleep(POLL_INTERVAL)
                poll = requests.get(
                    f"{COLAB_BASE_URL}/payload", timeout=10
                ).json()

                if poll["status"] == "pending":
                    continue
                if poll["status"] == "error":
                    _error(f"LLM error: {poll.get('error', 'unknown')}")
                    return
                if poll["status"] == "ok":
                    payload = _validate_payload(poll["data"])
                    bus.publish(payload)
                    _status(f"Ready: {payload.concept_title}")
                    return

            _error("Timeout: LLM did not respond in time.")
```

```
        except requests.ConnectionError:
            _error("Cannot reach Colab. Is the server cell still running?")
        except Exception as exc:
            _error(f"Bridge error: {exc}")

    threading.Thread(target=_worker, daemon=True).start()


def check_connection() -> tuple[bool, str]:
    """Synchronous health-check. Call once at startup."""
    try:
        r = requests.get(f"{COLAB_BASE_URL}/health", timeout=8)
        if r.json().get("status") == "alive":
            return True, "Colab server reachable."
        return False, "Unexpected health response."
    except Exception as e:
        return False, str(e)
```

New file: `chat_panel.py` ***(~170 lines)***

Self-contained Tkinter widget. Calls `llm_bridge.ask()` and receives the rendered explanation back from `gui.py`.

```
Public interface:
  ChatPanel(parent, on_status, on_error)
  .render_explanation(payload: VisualizationPayload)   ← called by gui.py
```

Internal responsibilities:

- Text input + Send button
- Scrollable conversation view (user / tutor bubbles)
- "Thinking…" state while polling
- Displays `payload.explanation` and `payload.equation` when result arrives
- Passes `on_status` to `llm_bridge.ask()` so the GUI status bar updates live

`gui.py` — 4 targeted changes

Change 1 — Wider window

```python
# BEFORE
self.minsize(1100, 720)
# AFTER
self.minsize(1440, 720)
```

Change 2 — Add chat column in `_build_ui`

```python
from chat_panel import ChatPanel

chat_col = tk.Frame(main, bg=PANEL, width=330)
chat_col.pack(fill="y", side="left", padx=(0, 8), pady=8)
chat_col.pack_propagate(False)
self.chat_panel = ChatPanel(
    chat_col,
    on_status=self._status,
    on_error=lambda msg: self._status(msg, error=True),
)
self.chat_panel.pack(fill="both", expand=True)
```

Change 3 — Start bus polling in `__init__`

```python
# after self._apply_preset(PRESETS[0][1])
self._poll_bus()
```

New method:

```python
```

```python
def _poll_bus(self):
    try:
        payload = viz_bus.subscribe().get_nowait()
        self._apply_llm_payload(payload)
    except Exception:
        pass
    self.after(200, self._poll_bus)
```

Change 4 — Apply payload (visualizer side only)

```python
def _apply_llm_payload(self, payload):
    """Load a Colab-emitted payload into the visualizer. No LLM code here."""
    self.eq_var.set(payload.equation)
    self._sliders["x_min"].set(payload.x_range[0])
    self._sliders["x_max"].set(payload.x_range[1])
    self._sliders["y_min"].set(payload.y_range[0])
    self._sliders["y_max"].set(payload.y_range[1])
    self._sliders["step"].set(payload.step)
    self._plot()
    self._status(f"Loaded: {payload.concept_title}")
    self.chat_panel.render_explanation(payload)
```

`main.py` — add connection check at startup

```python
from llm_bridge import check_connection

def main():
    ok, msg = check_connection()
    if not ok:
        print(f"[Warning] Colab server unreachable: {msg}")
        print("The visualizer will still open. Enter the URL in llm_bridge.py")
        print("and restart, or use the manual equation entry instead.")
    # rest unchanged
```

`requirements.txt` — 1 addition

```
requests>=2.31
```

Data-Flow Walkthrough

```
1. User types "explain the gaussian bell surface" → hits Send in ChatPanel.

2. ChatPanel calls llm_bridge.ask(text) → daemon thread starts,
   status bar shows "Sending question to Colab…"

3. Thread POSTs {"text": "..."} to https://xxxx.ngrok-free.app/ask
   Colab queues the generation job, returns {"status": "queued"}.

4. Thread polls GET /payload every 1.5 s.
   Colab generates:
   {
     "concept_title": "Gaussian Bell Surface",
     "equation": "exp(-(x**2 + y**2)/4)",
     "x_range": [-4.0, 4.0],
     "y_range": [-4.0, 4.0],
     "step": 0.18,
     "explanation": "The Gaussian bell surface ..."
   }
   /payload returns {"status": "ok", "data": {...}}

5. _validate_payload() checks all fields and ranges.
   viz_bus.publish(payload) deposits it onto the queue.

6. gui.py._poll_bus() fires on Tkinter main thread (after 200 ms),
   calls _apply_llm_payload(payload):
     • Sets equation entry  → "exp(-(x**2 + y**2)/4)"
     • Sets all domain sliders
     • Calls self._plot()  → surface renders in 3D canvas
     • Calls chat_panel.render_explanation(payload)

7. ChatPanel displays the explanation as a formatted tutor bubble.
```

Separation Guarantee

Module	Imports gui?	Imports llm_bridge?	Imports viz_bus?
`llm_bridge.py`	✗	—	✓ write
`viz_bus.py`	✗	✗	—
`chat_panel.py`	✗	✓	✗
`gui.py`	—	✗	✓ read
`main.py`	✓	✓ health only	✗

`gui.py` and `llm_bridge.py` are fully decoupled.
The LLM cannot call `_plot()`, cannot touch sliders, cannot trigger any GUI action.
It can only return JSON — the bridge validates it and the bus delivers it.

Summary of All Changes

File	Action	Est. lines
`viz_bus.py`	**Create**	~25
`llm_bridge.py`	**Create**	~80
`chat_panel.py`	**Create**	~170
`gui.py`	**Modify** (4 hunks)	~30
`main.py`	**Modify** (1 hunk)	~8
`requirements.txt`	**Modify** (1 line)	1
Colab notebook	**Create** (4 cells)	~120